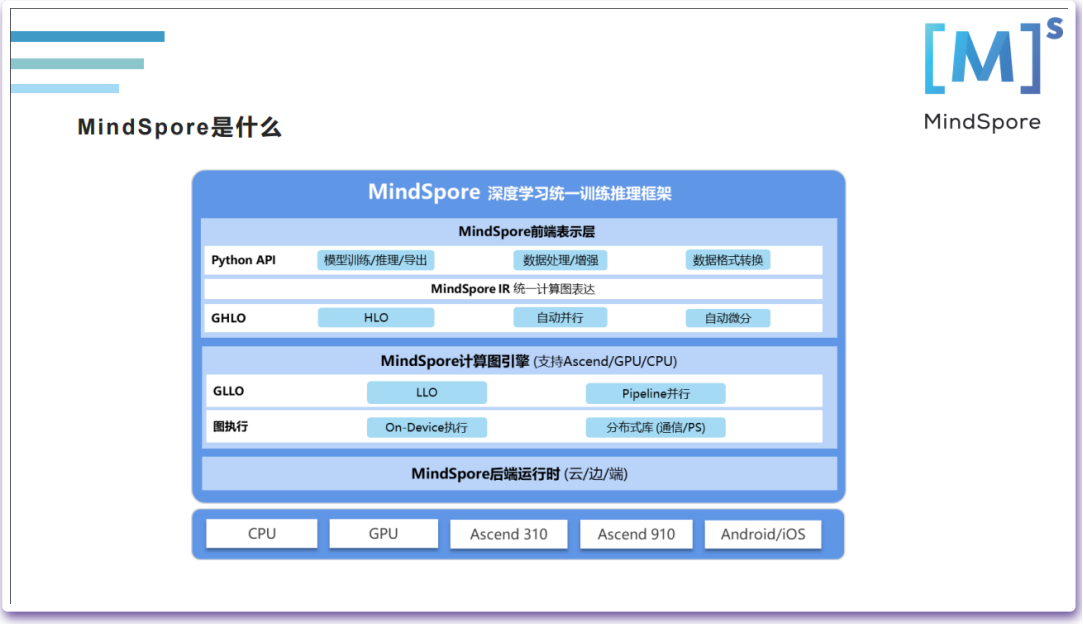
1 分布式自动并行训练

1.1 MindSpore是什么

- 深度学习统一训练推理框架

  框架总体分为三个部分

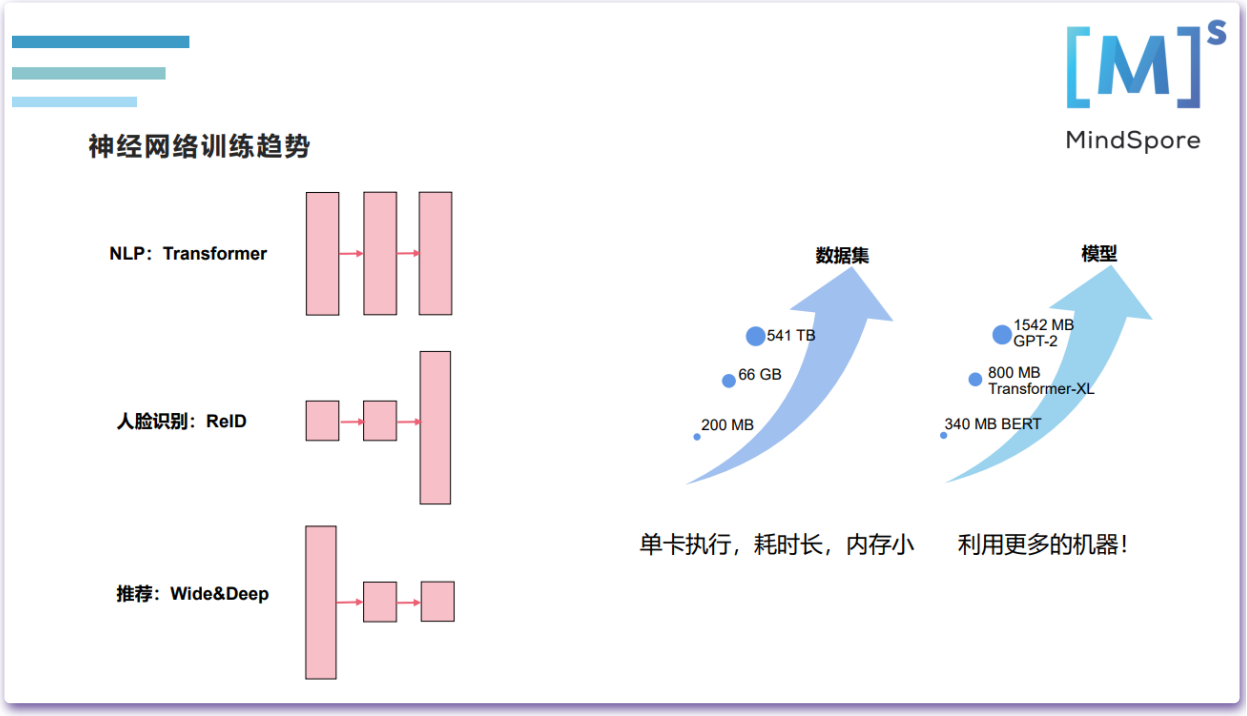
  1. 前端表示层
  2. 计算图引擎
  3. 后端运行时



1.2 并行模式概述

1.2.1 神经网络训练趋势

- 在神经网络发展过程中，为了提升网络学习能力和泛化能力，模型参数量和数据集的规模都呈现指数级的增长

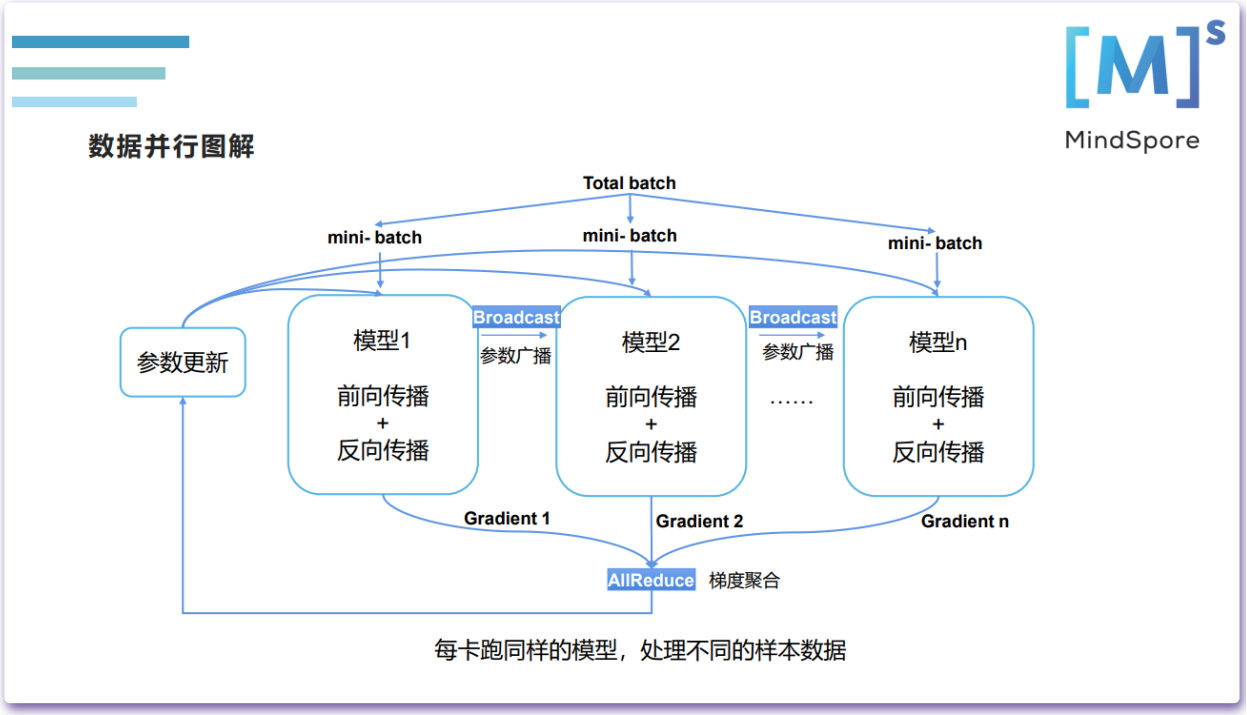


1. NLP模型通过堆叠transformer层获取更高的精度，随之带来的问题是单卡性能很快到达了性能上限
2. 人脸识别领域，卷积层的输出经过FC层后实现特征整合，而FC层的参数大小与图片分类类别数呈正相关，对于超大规模的人脸识别网络，必然会受到内存的制约
3. 推荐网络中，会出现同样的问题，由于特征太多造成头部embedding层非常大单卡无法容纳

- 为了突破内存限制, 在同样的时间内训练更多的数据, 分布式并行成为当下神经网络的训练趋势。目前主流的分布式训练范式有
    1. 数据并行
    2. 模型并行
    3. 混合并行

1.2.2 数据并行

- 数据并行图解



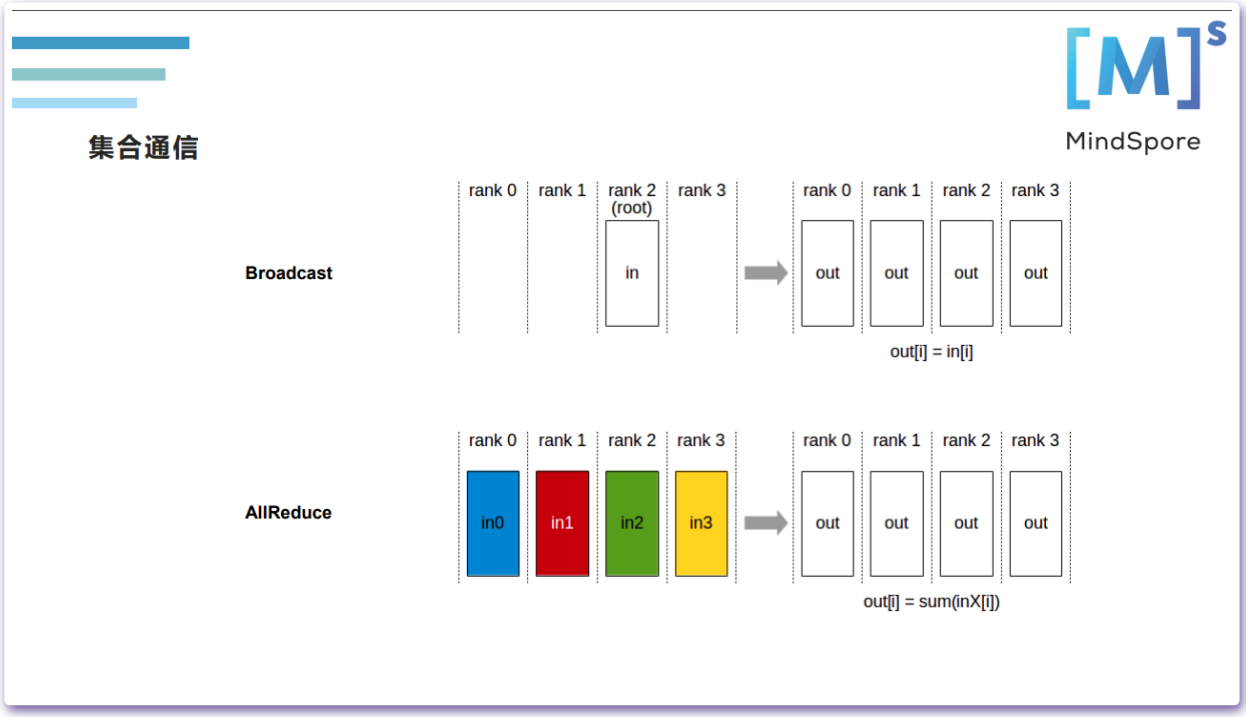
数据并行是当前业界应用最广泛的一种并行模式 ,它将模型在多台设备上进行复制,通过将样本数据分发到不同机器上执行的方式达到分布式并行的效果。假设每个批量160张图片，那么在8卡上面，每卡会分到20张图片进行训练，

这样 `对样本batch维度切分的方式我们称之为数据并行`

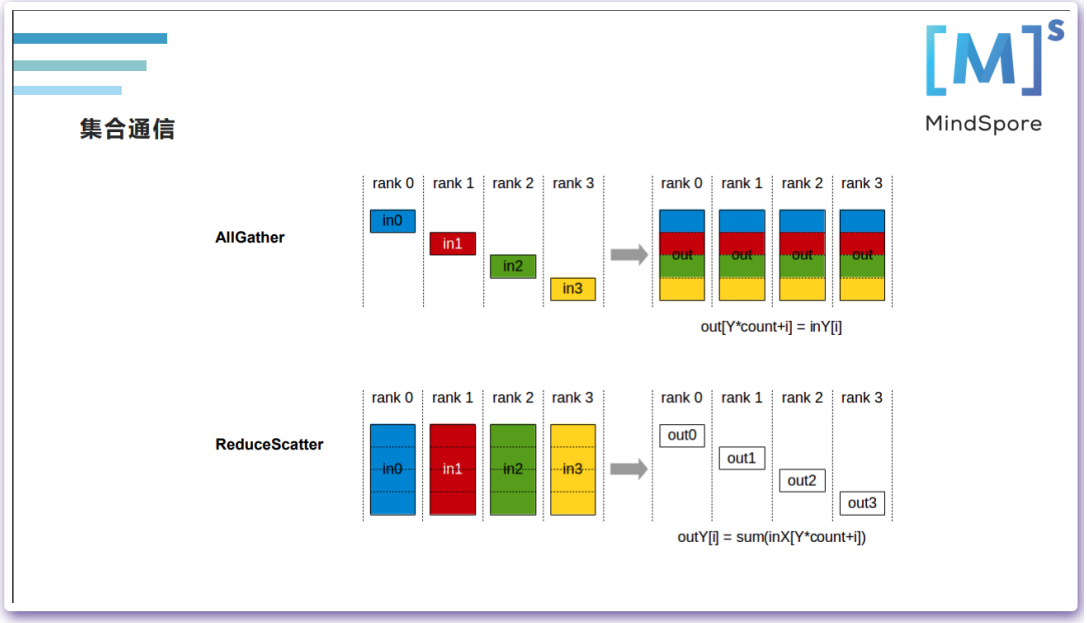
在每张卡上样本输入到网络后经过前向传播和反向传播的学习得到待更新的参数梯度,由于每张卡处理的样本不一致最终得到的梯度会有所差别,为了实现训练间的同步并和单机训练保持一样的数学逻,我们需要对每张卡上的梯度进行收集求和或者求均值计算后再执行权重更新操作,由此完成一轮的迭代训练。

从图上我们可以看到数据并行与单机执行的主要区别在于设备间的数据同步它也是所有分布式训练的基础

- 集合通信



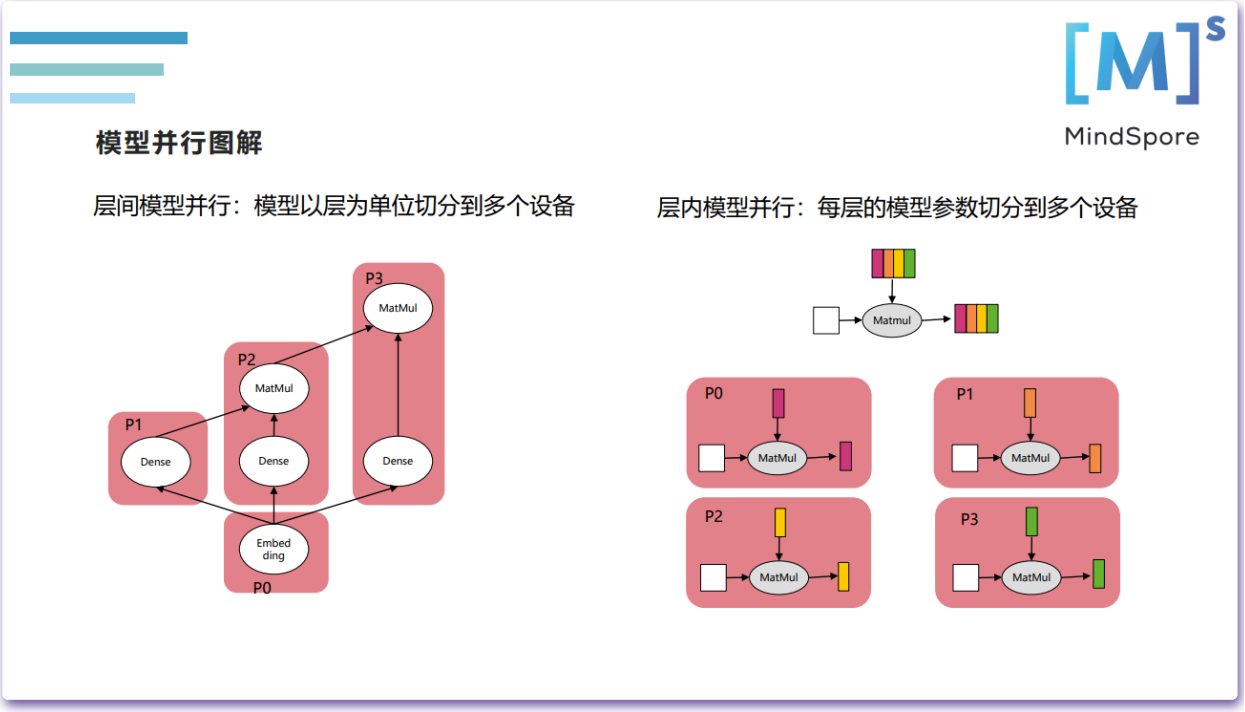
1. 模型复制，通过集合通信中的Broadcast操作实现，在参数初始化时将其中一台机器的数据发送到其他设备，最终每卡获得同样的数据
2. 梯度聚合 操作由**AllReduce**完成,它实现的是多对多通信,每台设备收集来自其他设备上的数据并把它们合并为单个数据项在图上,每台机器的输出等于所有设备上数据的求和,类似的例子还有求均值最大值最小值等。



3. `AllGather` 是将每卡的数据收集后在行方向上进行拼接，数据间的排列方式遵从设备的排序
，也就是说rank0上的数据会出现在第0行rank1上的数据会出现在第一行以此类推。
4. `ReduceScatter` 可以看作是两种通信方式的叠加，首先执行AllReduce 的sum操作将每卡的数据求和。再将结果在行方向上进行切分根据rank号分发到不同的设备上

1.2.3 模型并行

- 数据并行的优势在于同样的时间下可以训练更多的数据量，然而对于超大规模网络数据并行，无法解决模型内存过大的问题需要通过模型并行来实现训练
- 模型并行图解



模型并行有层间和层内两种切分方式

1. 层间模型并行：将模型以层为单位切分到多个设备,每卡执行的网络结构存在差异
2. 层内模型并行:不改变网络结构,而是将每层的模型参数切分到多个设备上实现并行。
   以图上这个例子而言,将Matmul算子的权重在列方向上进行了切分.拆开到不同的卡上计算,每卡保存全量结果其中一个分片。

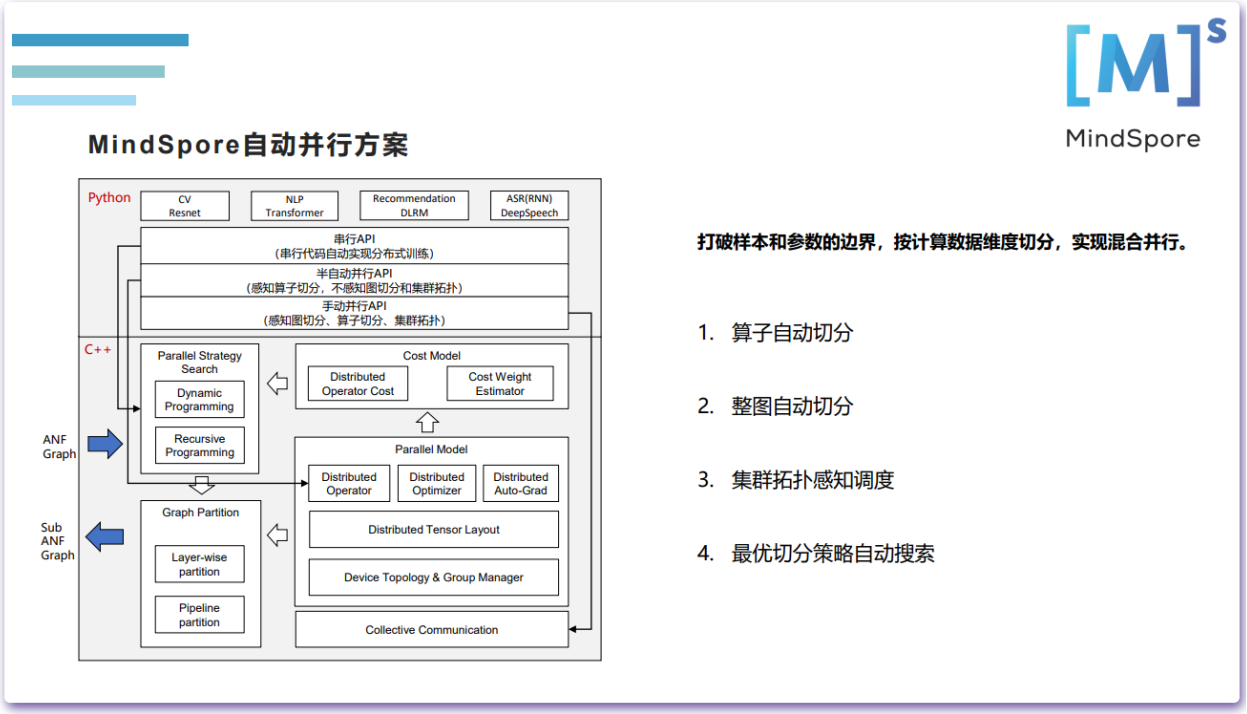
- 相比数据并行模型并行的难度更大，主要体现在三个方面
  1. 内存上限：首先开发者需要考虑内存上限，让切分后的子模型同样能够在单卡中运行
  2. 通信代价：其次在性能上要兼顾通信开销让网络的整体性能有所提，也就是集合通信操作产所产生的耗时
  3. 张量排布：最后开发者还需要关心模型切分后张量存在的形式是否为切片以及切分的维度。

  在不破坏原有数学逻辑的原则下进行正确的分布式训练。

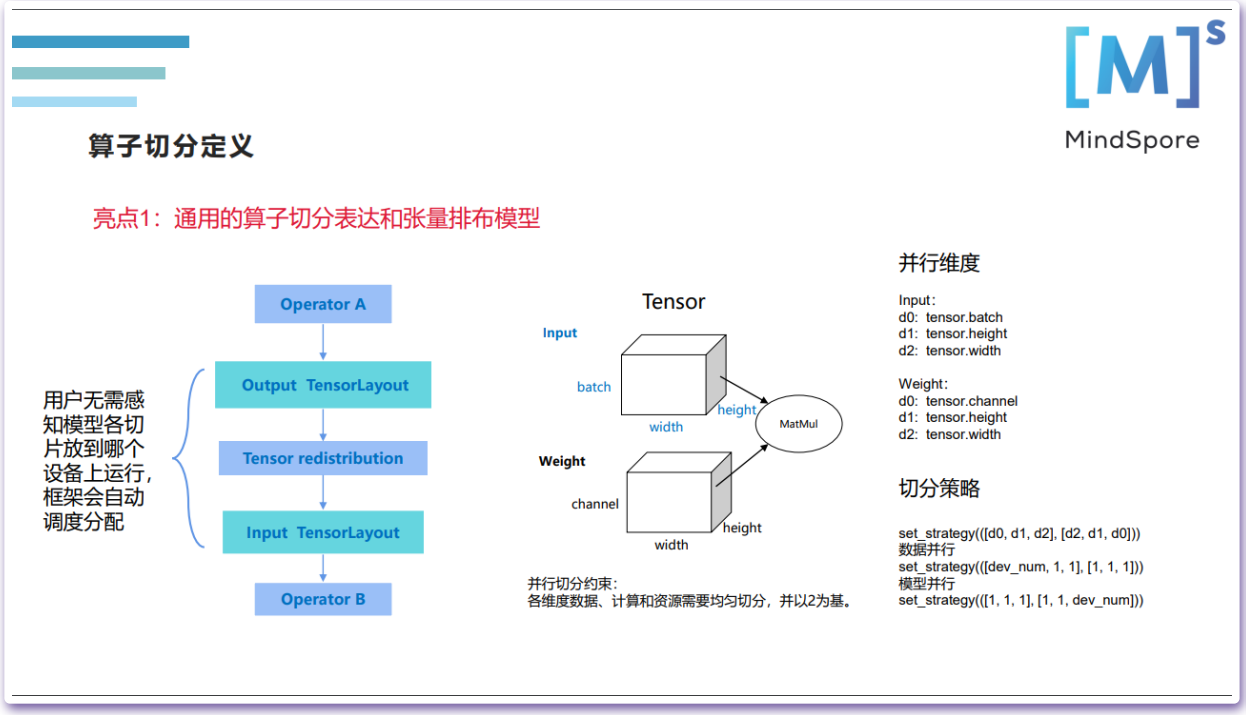
1.3 自动并行架构介绍

1.3.1 mindspore自动并行

- mindspore自动并行方案



- 算子切分定义



看一下右边这个三维矩阵乘的例子,输入有batch width height3个维度,权重有channel white height3个维度,

其中1表示不切分，dev_num表示把数据均分到每台设备上。

```
set_strategy(([d0, d1, d2], [d2, d1, d0]))
```

1. 数据并行

```
set_strategy(([dev_num, 1, 1], [1, 1, 1]))
```

在input的 batch维度上均分（d0位置为batch维度），指定它为device number，其他维度上不切分

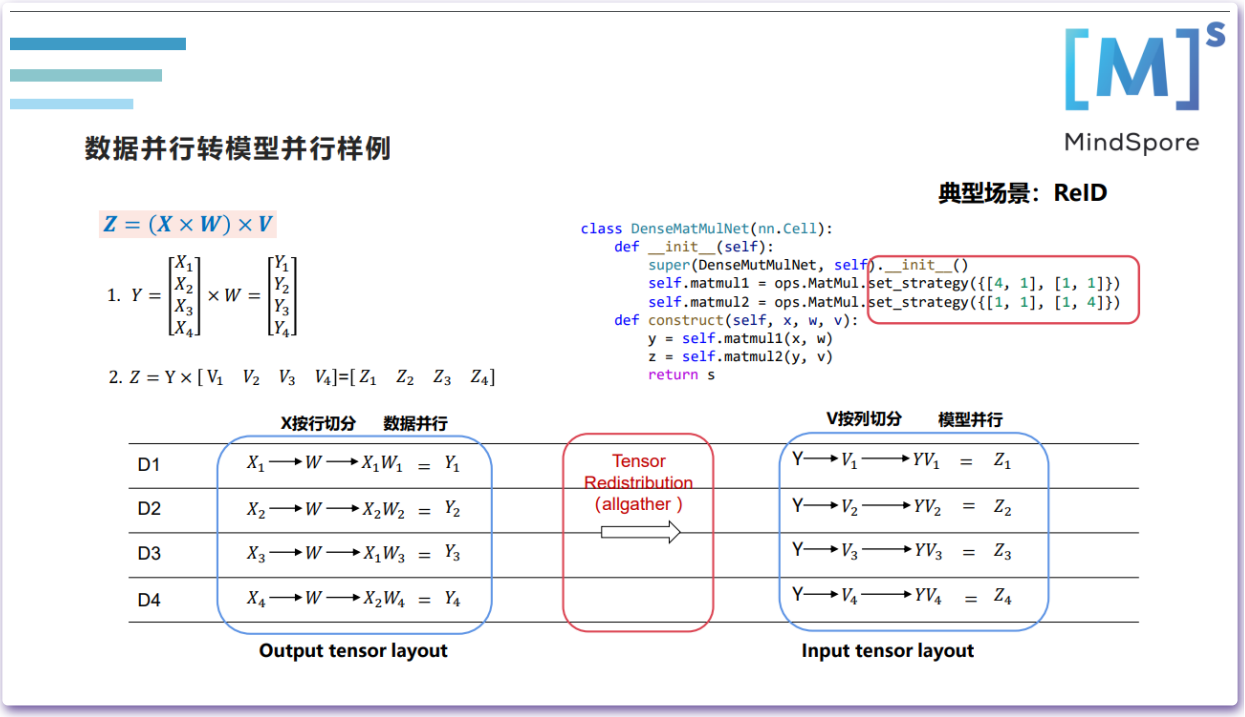
2. 模型并行

```
set_strategy(([1, 1, 1], [1, 1, dev_num]))
```

在weight的witch维度上均分（d2位置上为width维度），指定它为device number，其他维度上不切分

1.3.2 混合并行实例（半自动并行,二维矩阵乘法示例）

- 数据并行转模型并行



1. 第一个矩阵乘算子，对应 $X \times W$ 的操作

```
self.matmul1 = ops.MatMul.set_strategy({[4, 1], [1, 1]})
```

表示对第一个输入X在行向量上切分为4份,列方向不指定，对第二个输入W行列都不指定

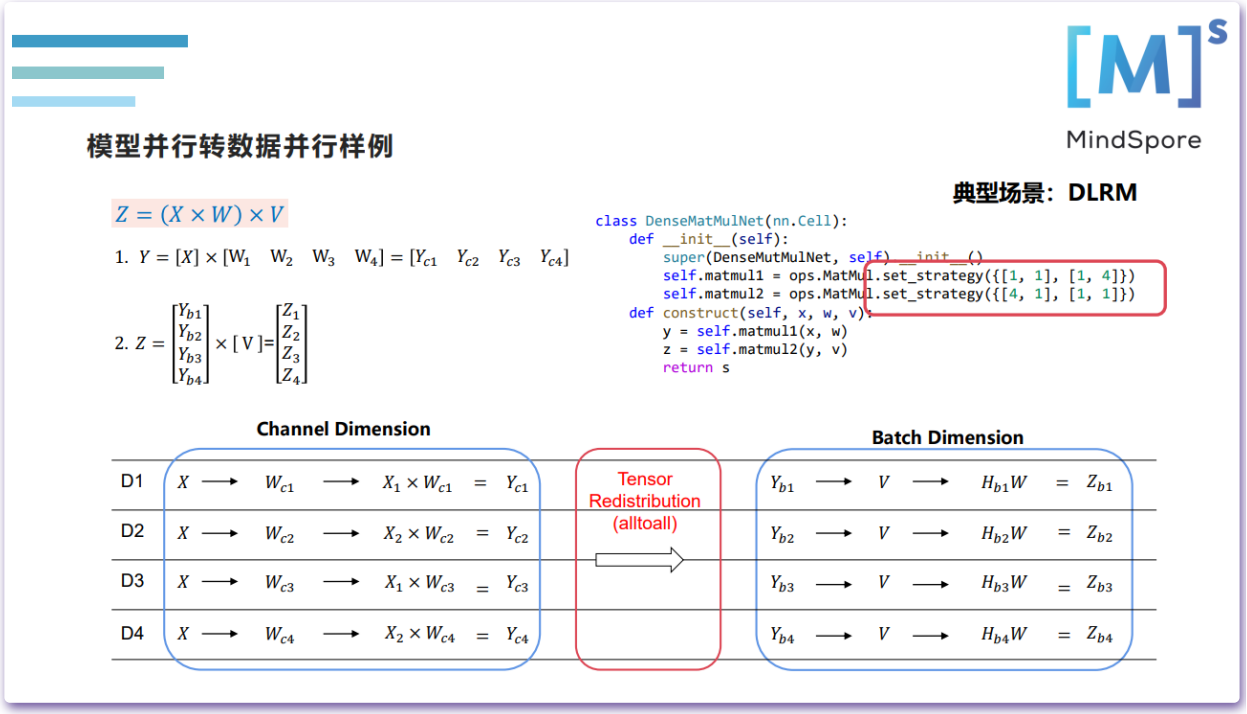
2. 第二个矩阵乘算子，对应 $Y \times V$ 的操作

```
self.matmul2 = ops.MatMul.set_strategy({[1, 1], [1, 4]})
```

表示对第一个输入Y不指定，对二个输入V在列向量上切分为4份

在第一个算子输出和第二个算子输入之间存在一个集合通信的操作，需要将行方向上切分的Y转为一个全量的Y，框架会自动推导出来插入一个allgather算子嗯，实现Y在行方向上的拼接嗯

- 模型并行转数据并行



1. 第一个矩阵乘算子，对应 $X \times W$ 的操作

```
self.matmul1 = ops.MatMul.set_strategy({[1, 1], [1, 4]})
```

表示对第一个输入X不指定，对二个输入W在列向量上切分为四份，行向量不指定

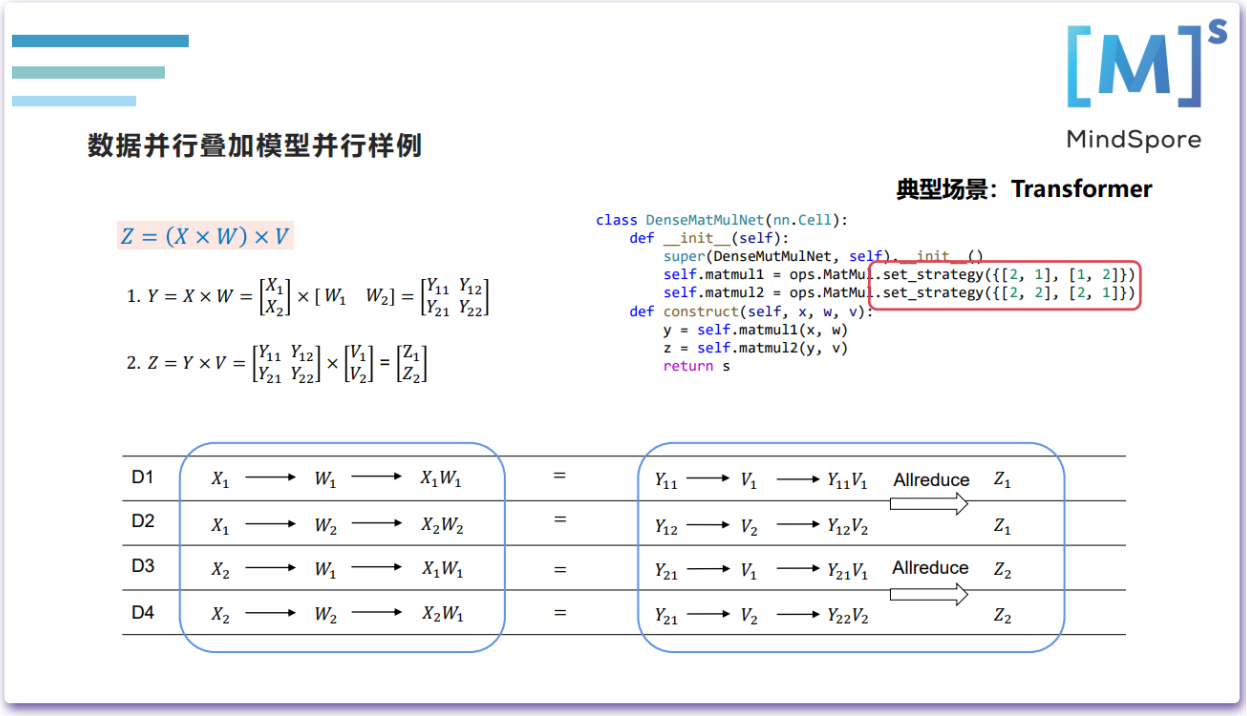
2. 第二个矩阵乘算子，对应 $Y \times V$ 的操作

```
self.matmul2 = ops.MatMul.set_strategy({[4, 1], [1, 1]})
```

表示对第一个输入Y在行向量上切分为4份,列方向不指定，对二个输入V不指定

在第一个算子输出和第二个算子输入之间存在一个重排布，对列方向上切分的Y，需要把它转化为行方向上切分的Y，这里需要插入一个类似矩阵转置的操作。在集合通信上我们称它为alltoall操作，·这样可以实现一个张量模型的转换，同样也是由框架自动插入的。

- 数据并行叠加模型并行



1. 第一个矩阵乘算子，对应 $X \times W$ 的操作

```
self.matmul1 = ops.MatMul.set_strategy({[2, 1], [1, 2]})
```

表示对第一个输入X行向量上切分两份，对二个输入W列方向上切分两份

2. 第二个矩阵乘算子，对应 $Y \times V$ 的操作

```
self.matmul2 = ops.MatMul.set_strategy({[2, 2], [2, 1]})
```

表示对第一个输入Y在行列方向都切分两份，对二个输入V只在行向量方向上切分两份

在第一个算子输出和第二个算子输入之间，这里的张量排布方式相同，不需要插入任何重拍布。但是由于在第二个矩阵乘算子两个输入的相关维进行了切分，所以在这个计算结果相应的分块上，我们需要插入Allreduce操作，
把Y1V1和Y12V2相加能得到正确的一个Z1这个结果。

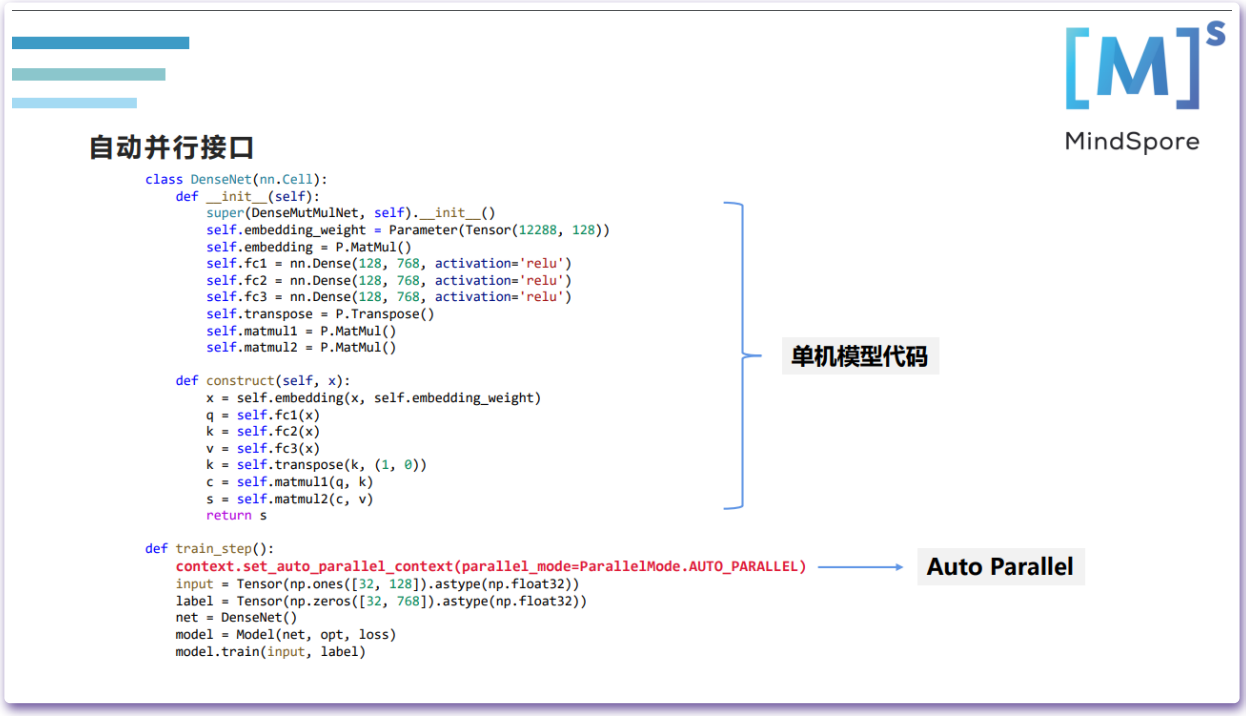
1.3.3 策略搜索原理（自动并行）

- 高效的并行策略搜索算法

  围绕升腾910芯片基于内存的计算开销和通信开销，对训练时间建模计算出不同数据量、不同算子在不同切分策略下近似的训练时间，然后通过自研的动态规划算法自动搜索出在一定内存阈值下性能最优的切分策略，这一步骤代替了用户手动指定模型切分，在较短的时间内得到高性能的切分方案降低了运行并行训练的使用门槛

1.4 Live Demo

- 自动并行接口



1. 类的初始化函数中定义需要用到的算子，变量，网络单元
2. 在construct网络中定义正向网络的连接方式
3. 通过Model接口将网络优化器，损失函数等封装后就可推理、训练了

- ResNet50网络数据并行、自动并行分布式训练编程实践

2 GraphEngine

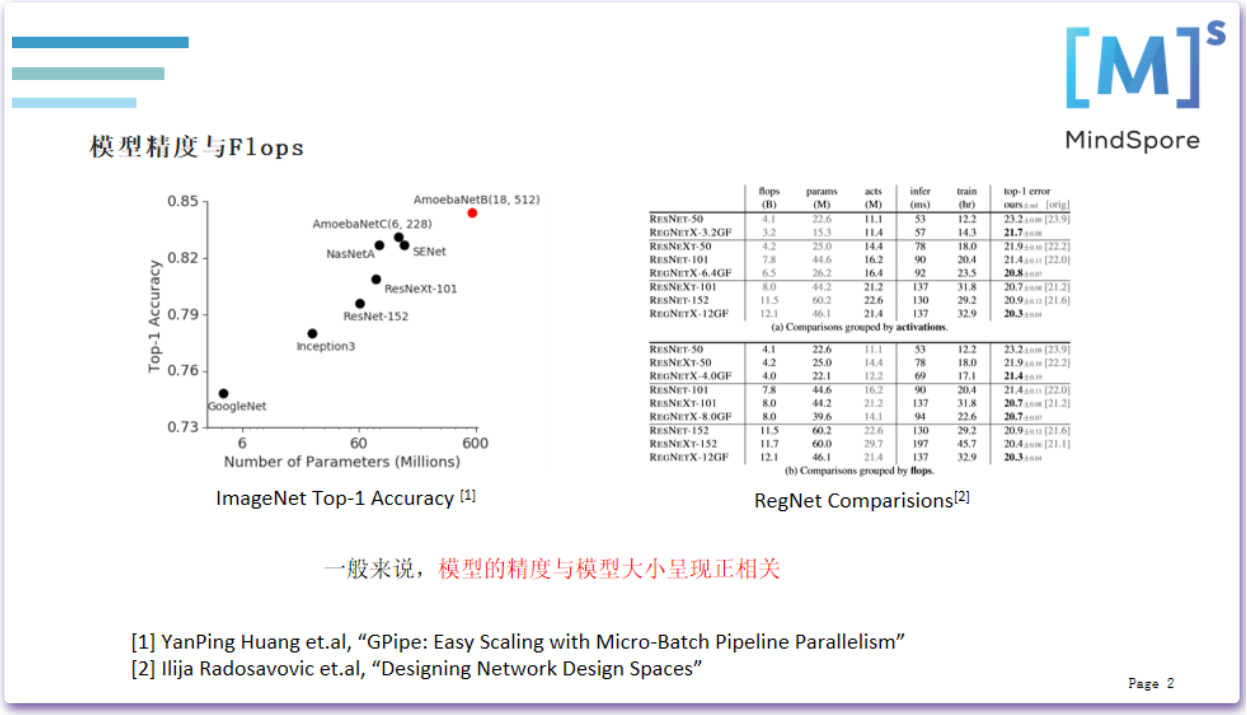
- ，GE专门负责和Ascend芯片交互。

2.1 芯片

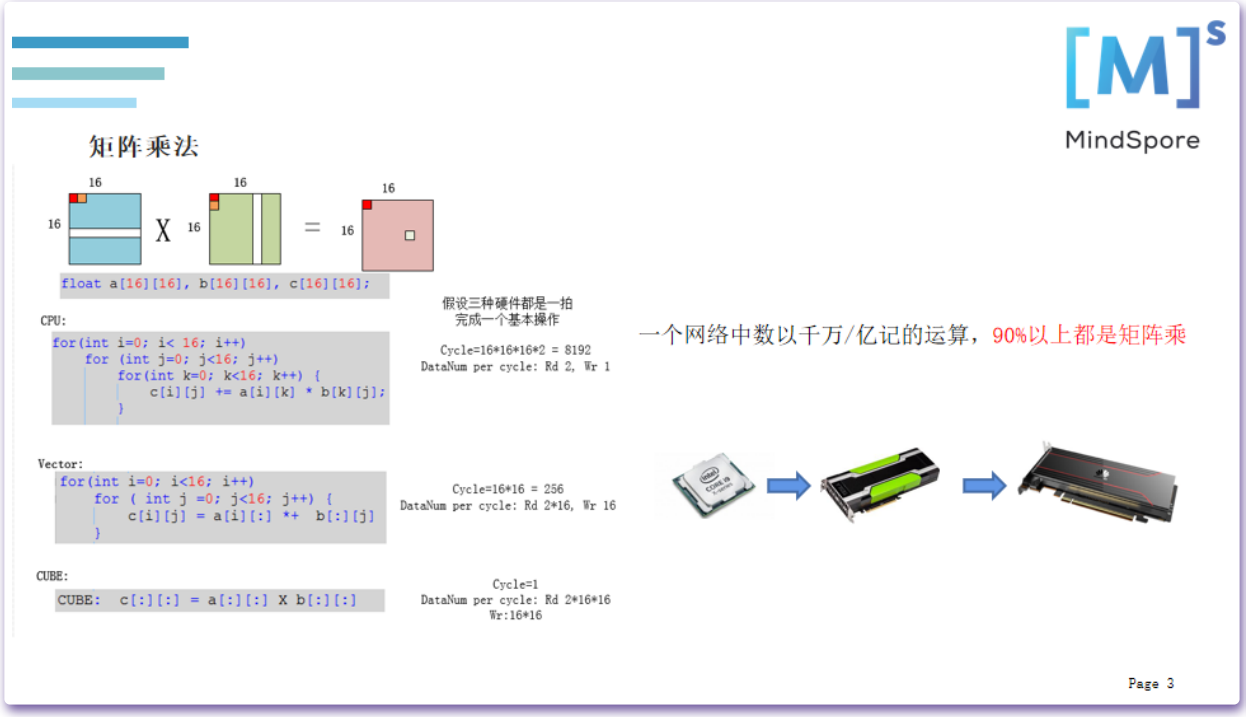
- 深度学习芯片简介（18年的图）

四个象限，上面是训练芯片，下面是推理芯片

mindpore原生态支持Asend芯片

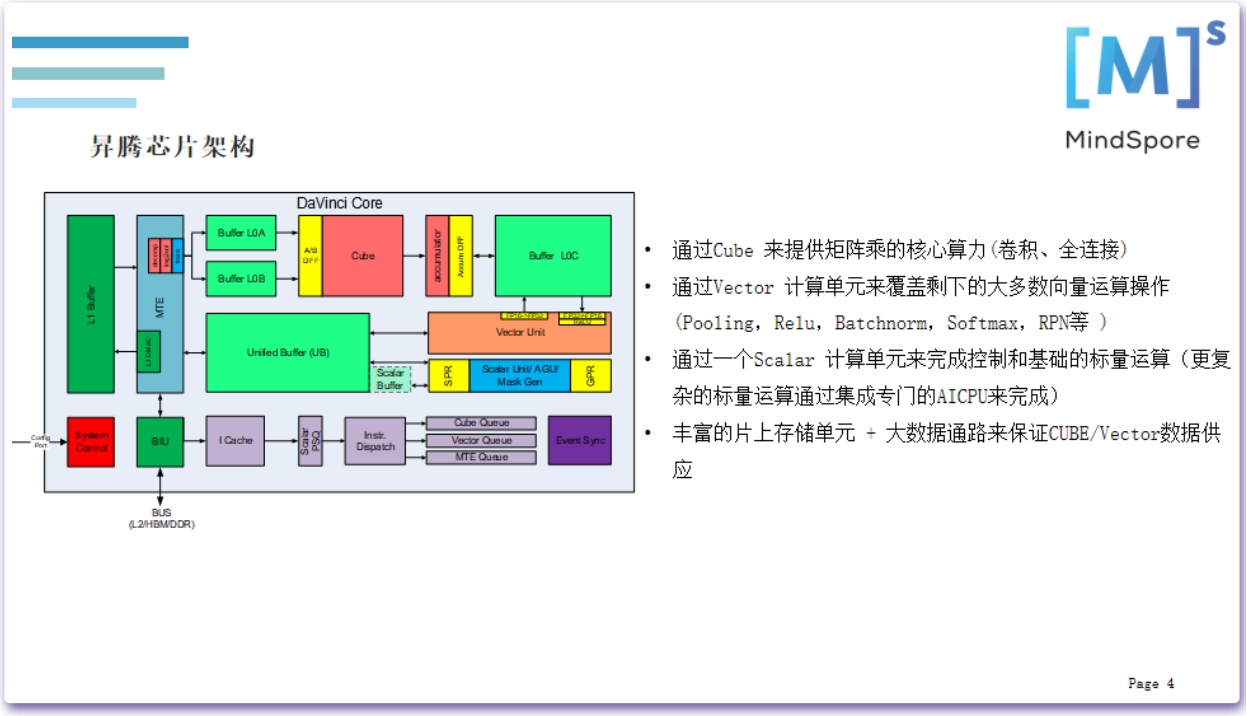


accuracy越高，需要的参数也原来越多，一般来说当一个模型可以被比较好的训练的时候，模型的精度和模型大小是呈现一个正相关关系的。



在一个网络数以亿计的运算当中90%以上都是矩阵乘操作，此时如何高效地去解决一个矩阵乘的运算，就会变成如何提高硬件运算能力的一个关键地方。

以上图16*16的矩阵运算，以CPU来看，一拍只能去做一个乘法或者一个加法运算，需要8192个时钟周期；如果操作对象是一个vector，一个行向量和一个列向量的话我总体需要256拍；但是如果基本运算单元就是一个16×16的矩阵那么我完成这样一个操作就只需要一拍，这也就是升腾910所采用的一种策略。



2.2 GraphEngine

- `GE` :GraphEngine(图优化引擎)
- mindspore架构图

1. On-Device Execution

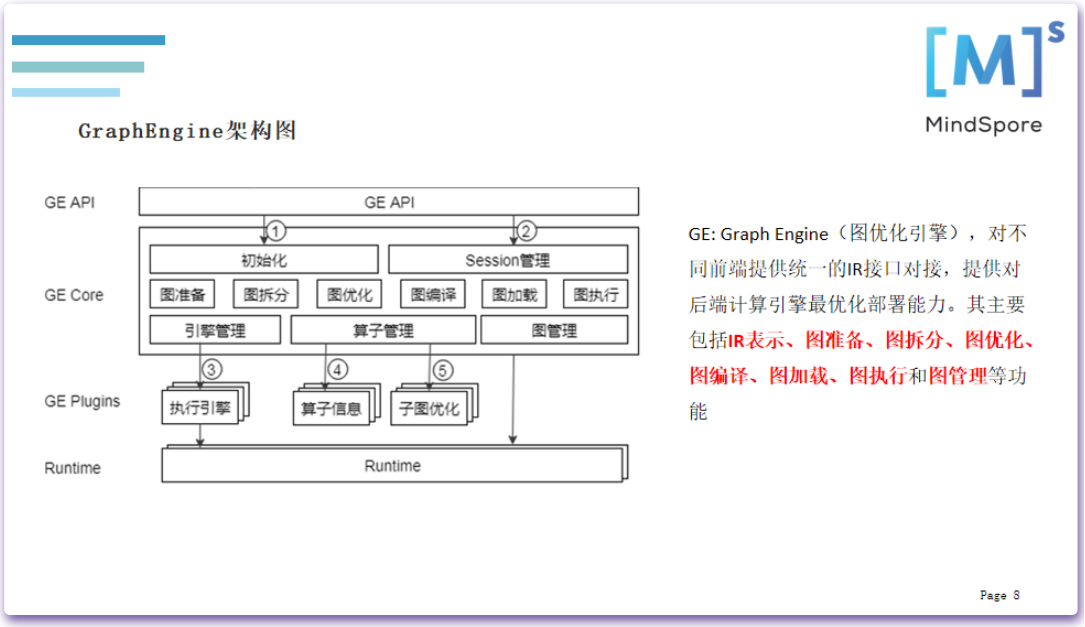
   整图下沉策略，可以将整张的计算图全部都放到芯片上去执行，减少芯片和host之间的一个数据交互的次数。

2. Pipeline Parallel

与之前自动并行中的定义的Pipeline Parallel（层间模型并行）不同，这里指的就是单纯的一些并行的计算而已，比如分布式读取数据，部分的并行反向计算等。

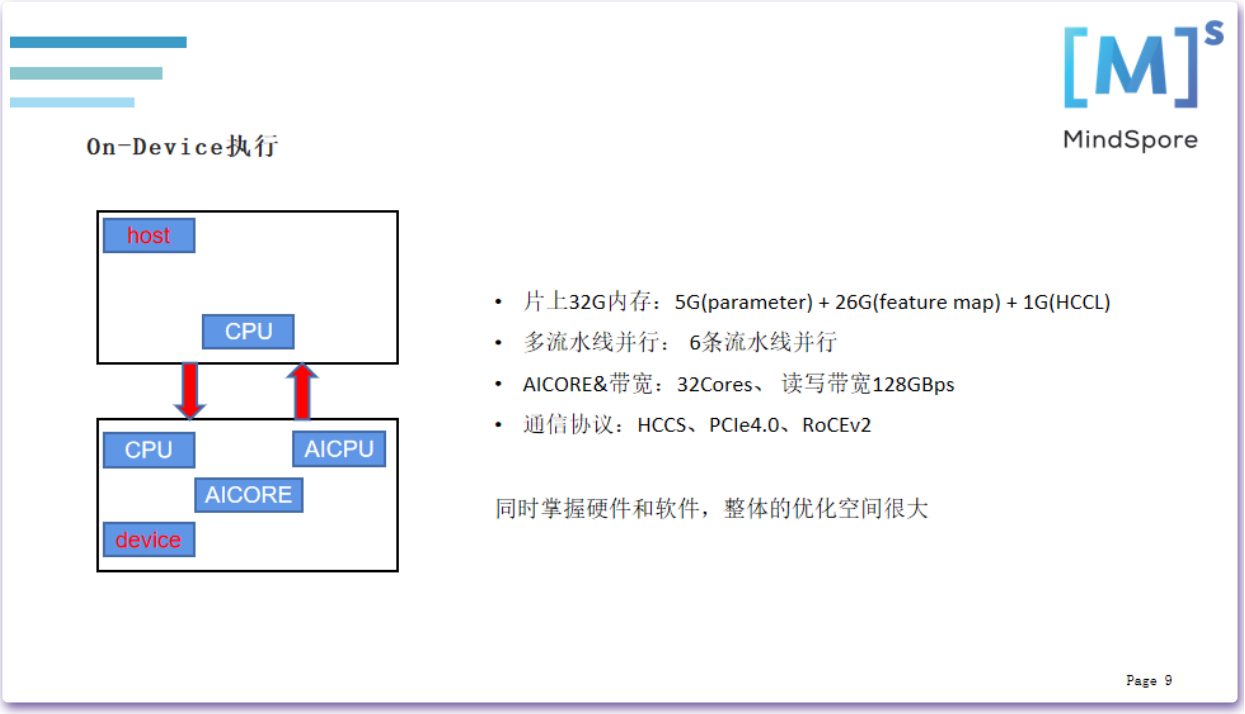
3. Low level Optimization

   针对升腾芯片做的图优化工作



2.2.1 on-device-execute

- on-device



上图左边，上面是host，下面是device，如果下面只有AICORE的话，那么流程就是，cpu负责逻辑控制，将任务分发到device，device对该图进行运算，若某些标量不适合这个tensor-enging处理，会希望啊host CPU去完成这些运算，发回CPU，算完后再返回device继续剩下的运算，不断往复，会会频繁的在host和device之间进行交互。

而升腾芯片上配备了AICORE: 大型tensor的vector运算，配备了AICPU可以进行稍微复杂点的标量运算，同时配备了一个相对小一点的cpu，但是足够应付深度学习场景下的逻辑控制分发的任务。

所以他就会将整图一次性的下发给生成芯片，然后由这一块生存芯片小CPU开始完成整个任务调度和整个的逻辑分配工作，在片上完成了整个图的运算，最后我们返回结果返回给CPU。

为了保证整体的这个on-device执行，配备了一个相对比较大的内存。默认分配如上图（HCCL算子用于通信）。

32个AICORE，每个读写带宽128G。

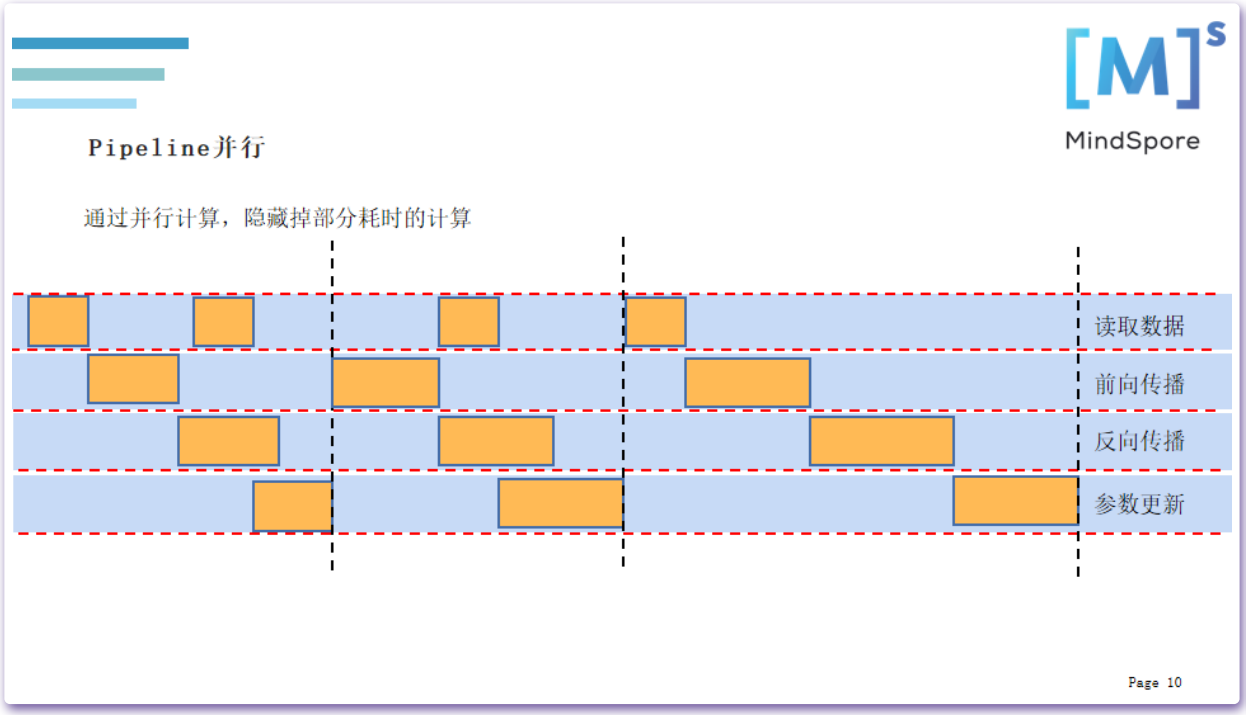
六条流水线包括SCALER流水线vector和cube的流水线，还有三条数据搬运的流水线，可以一定程度上并行。

自研了一个HCCS的通信协议，同时配合使用PCIE4.0，RoCEv2通信协议。

整体上我们这些所有的策略，都是想保证一个足够大的数据通路，能够让时间更多地被使用在数据的计算当中，而不是数据的搬运上面。

2.2.2 Pipeline并行

- 通过通过一定程度的并行计算，来隐藏掉部分相对比较耗时的计算。



原理：

4条管道，读取数据（cpu操作），前向传播，反向传播，参数更新，如果没有并行的话，如第三个，读取完数据后才能进行前向传播，前向传播完成后才能进行反向传播，但是参数更新操作(广义，包括在DDP情况下从不同卡做梯度聚合的操作和参数更新的操作)并不一定要等到反向传播完全算完才能开始。

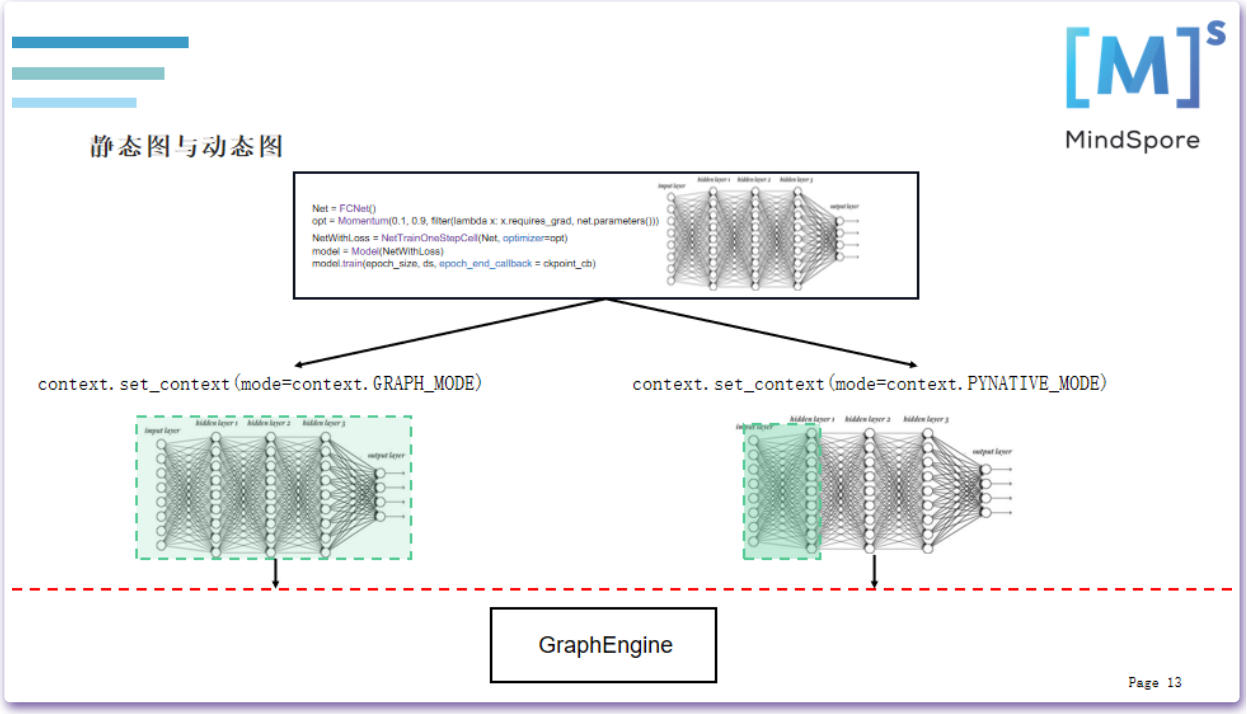
如上图第一个，假设网络中有100个参数，我们算完了前50个参数对应的梯度，就可以开始

不同卡上的这前50个梯度的聚合，这不影响之后的反向过程，这时我们cpu相对空闲，可以去读取数据（其实就是一个分布式的数据读取读取操作），等到下一个周期开始的时候就可以直接开始前向传播。

2.2.3 Low level Optimization

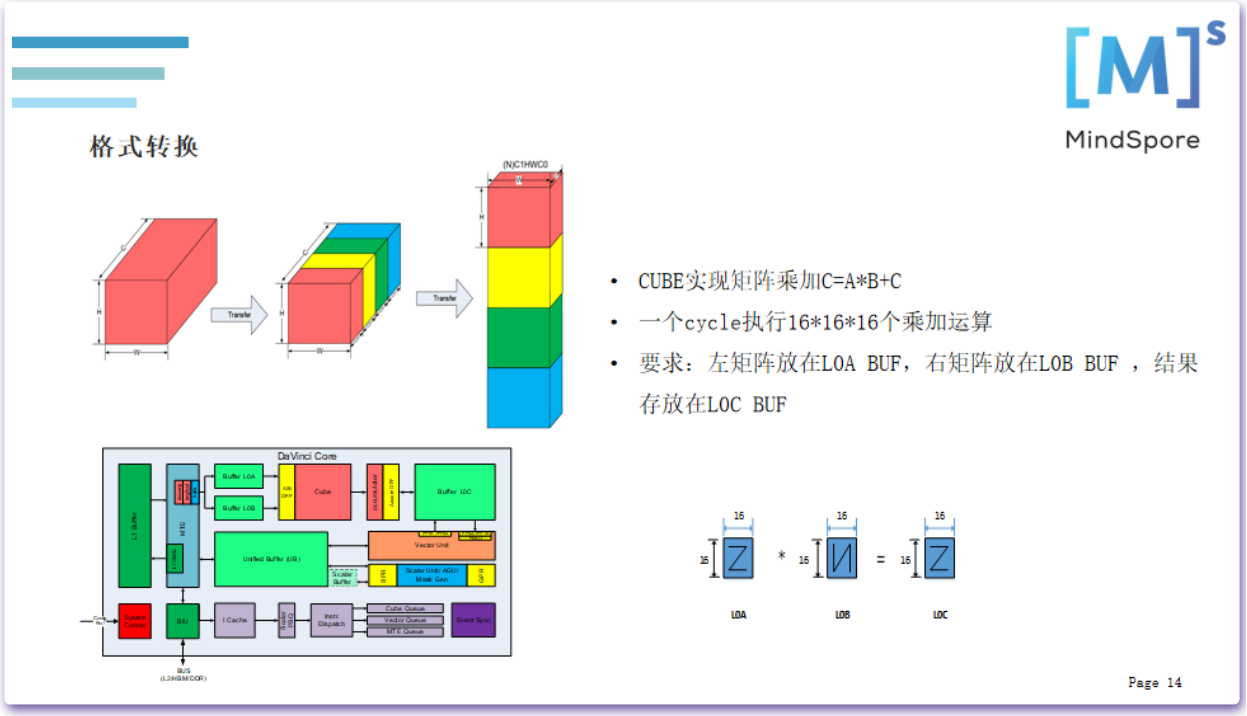
- 深度图优化

- 静态图与动态图

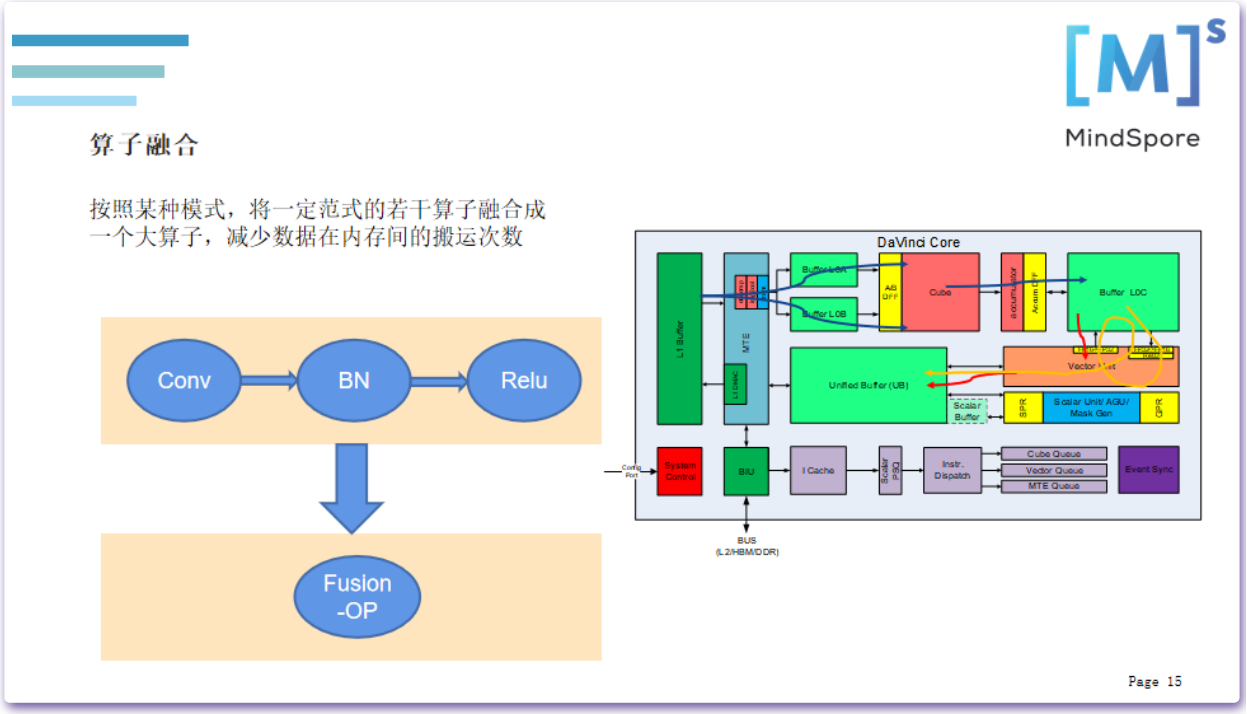


graph mode就是静态图，pynative mode就是动态图，机器无法感知，只是单纯拿着数据流图作为自己的输入，然后走完上面的图准备、图拆分等等6个步骤。使用这个pynative mode的话,它势必会比这个graph mode要更慢点更繁琐一点,因为要不停的去做这6个深度图优化的流程。但是本身动态图和静态图就是这样一种在效率和灵活性之间的一种权衡工作。

- 格式转换



- 算子融合



2.3 debug工具

- 数据dump

通过设置一个环境变量指向一个你所定义的一个文件，然后你需要声明一个环境变量指向一个你自己定义的JSON文件。

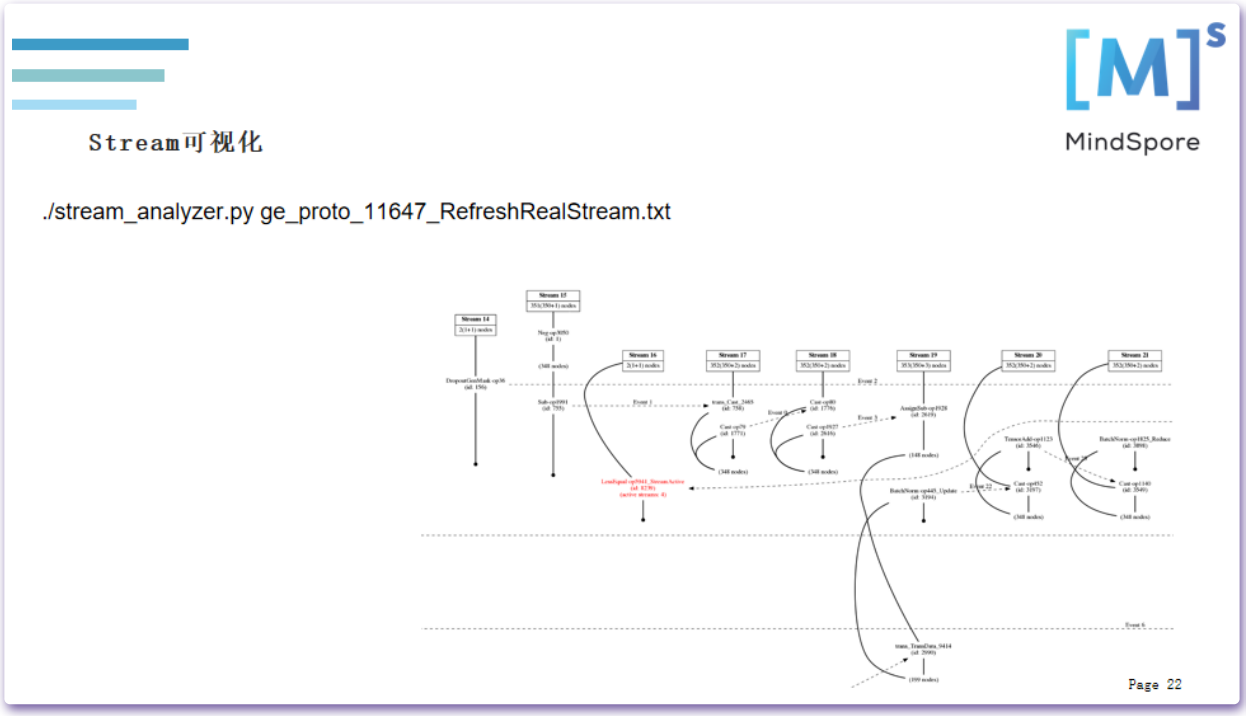
通过配置可以dump出任何存在在网络中的数据，每一个算子的输入输出，每一个

parameter的值和梯度都可以被dump下来。

- 图dump

- profiling（只能在升腾芯片上用）

- Stream可视化



2.4 升腾芯片性能（2020）

- Atlas900训练集群

  由1024块升腾920芯片组成的训练集群，使用mindspore这做一个reset50的训练,在IMAGENET上训练90个epoch，整体的耗时只需要59.8秒。